

Toast Personalizado

Se comenzará por hacer un `Toast` genérico en este proyecto para posteriormente pasar a un diseño más personalizado. Un `Toast` es una ventana emergente que se muestra unos segundos y que normalmente tiene un texto, aunque, como se verá, se puede personalizar dicho aspecto.

Para ello se comenzará y partiendo del proyecto creado en el diseño de pesos habrá que poner un identificador al botón `Toast` editándolo en el archivo `activity_main.xml`:

```
<ImageButton
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="3"
    android:background="@drawable/boton_pulsado"
    android:id="@+id/btnToast" />
```

En `MainActivity.java` se implementa un `Listener`, que consiste en una clase que gestionará el click del botón. Para ello, en primer lugar se captura la referencia al botón en concreto a través de su identificador y posteriormente se utiliza la función `setOnClickListener`. Dentro de dicha función se instancia la clase `OnClickListener` y en su función `onClick` se implementa lo que se quiera que se haga con el click, en este caso lanzar un `Toast`. Como parámetros se pasarán:

- Contexto: es la forma de reflejar al `Toast` la actividad en la que hay que lanzar la ventana emergente.
- String: el texto que se quiere mostrar.
- Tiempo: la duración que el `Toast` tendrá en pantalla. En este caso se usa un atributo estático que contiene la clase `Toast` para reflejar un periodo de tiempo corto.

```
package com.example.disenyopesos;

import android.os.Bundle;
import android.view.View;
import android.widget.ImageButton;
import android.widget.Toast;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        ImageButton btnToast = findViewById(R.id.btnToast);

        btnToast.setOnClickListener(new
        View.OnClickListener() {
```

```
@Override
public void onClick(View v) {
    Toast.makeText(getApplicationContext(),
        "Sergio y Ander", Toast.LENGTH_SHORT).show();
    }
});
}
```

A partir de este momento se comenzará a lanzar una ventana emergente personalizada. Para ello se agregará el logo de Almi al proyecto. La forma más sencilla de realizarlo es copiar la imagen y en la vista de proyecto de Android Studio copiarla en la carpeta drawable.

Para la creación del archivo XML habrá que hacer clic derecho en `res/layout` y hacer `new->layout resource file`, dando al archivo el nombre `toast_per.xml`. Dentro del archivo habrá que componer la siguiente interfaz:



El código resultante tendrá el siguiente aspecto

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/layoutToast"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <ImageView
        android:id="@+id/ivToast"
        android:layout_width="200dp"
        android:layout_height="200dp"
        android:src="@drawable/logo_almi" />

    <TextView
```

```

        android:id="@+id/tvToast"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Almi"
        android:textSize="40sp"
        android:gravity="center" />
    </LinearLayout>

```

En este diseño se opta por crear un `LinearLayout` con un ID. Dentro del mismo se crea un `ImageView` con id y una imagen. Así mismo se crea un `TextView` al que se asignará un texto que posteriormente será modificado.

En `MainActivity.java` se modifica el código para dejarlo con el siguiente código

```

package com.example.disenyopesos;

import android.app.Dialog;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.view.View;
import android.widget.ImageButton;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        ImageButton btnToast = findViewById(R.id.btnToast);

        btnToast.setOnClickListener(new View.OnClickListener()
        {
            @Override
            public void onClick(View v) {
                // Se infla el layout personalizado
                View vista =
                getLayoutInflater().inflate(R.layout.toast_per, null);

                // Se personalizan la imagen y el texto
                ImageView imagen =
                vista.findViewById(R.id.ivToast);
                imagen.setImageResource(R.drawable.andando1);
                TextView texto =
                vista.findViewById(R.id.tvToast);
                texto.setText("Ander y Sergio");

                // Se crea el diálogo con esa vista
            }
        });
    }
}

```

```

        final Dialog dialogo = new
Dialog (MainActivity.this);
        dialogo setContentView(vista);
        if (dialogo.getWindow() != null) {

dialogo.getWindow().setBackgroundDrawableResource(android.R.co
lor.transparent);
        }
        dialogo.show();

        // Se cierra solo a los 2 segundos
        new
Handler (Looper.getMainLooper()).postDelayed(new Runnable() {
            @Override
            public void run() {
                dialogo.dismiss();
            }
        }, 2000);
    }
}
}
}

```

En el código anterior se parte de inflar el diseño personalizado. El inflador ejecuta el método `inflate`, que recibe como primer parámetro el archivo XML del que toma la vista y como segundo parámetro `null`, dado que la vista no necesita un contenedor padre para mostrarse en un diálogo. Una vez inflada la vista, se personalizan sus elementos: mediante `setImageResource` se sustituye la imagen y mediante `setText` se cambia el texto.

A continuación se declara el `Dialog` y se le asigna la vista inflada con `setContentView`. Es importante fijar el fondo del diálogo como transparente con `setBackgroundDrawableResource`, ya que de lo contrario se mostraría un recuadro blanco alrededor del contenido y no se apreciaría como una ventana flotante. Con `show()` se muestra el diálogo en pantalla.

Para que el diálogo desaparezca solo, imitando la duración de un `Toast`, se utiliza un `Handler`, que ejecuta el cierre (`dismiss`) transcurridos los milisegundos indicados (en este caso 2000, es decir, 2 segundos).

Cabe destacar dos aspectos. El primero es que el `Dialog` se crea pasándole `MainActivity.this` como contexto: al estar dentro de una clase anónima, la referencia `this` haría alusión al propio `listener` y no a la actividad, por lo que hay que indicar la actividad de forma explícita. El segundo es que la variable `dialogo` se declara como `final`, dado que se utiliza dentro de otra clase anónima (el `Runnable` del `Handler`); toda variable que se emplee dentro de estas clases debe ser declarada como `final`.